



Clase 43 - Instalaciones básicas para Docker y Kubernetes

Para poder instalar Docker y Kubernetes debes tener instalada tu maquina virtual de Ubuntu Server 24.04.4. La ISO se descargó a principios del curso.



O bien:

```
ip a
```

Ejemplo de salida:

```
192.168.86.130
```

2. Verificar que SSH está instalado

Comprobar si el servicio SSH está funcionando:

```
sudo systemctl status ssh
```

Si aparece:

```
Active: active (running)
```

SSH está correctamente instalado.

Para salir de la pantalla:

```
q
```

3. Instalar SSH (si fuera necesario)

Actualizar repositorios:

```
sudo apt update
```

Instalar OpenSSH Server:

```
sudo apt install -y openssh-server
```

Habilitar el servicio:

```
sudo systemctl enable ssh
```

Iniciar el servicio:

```
sudo systemctl start ssh
```

Comprobar nuevamente:

```
sudo systemctl status ssh
```

```
curso@dataeng:~$ sudo systemctl enable ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
Created symlink /etc/systemd/system/ssh.service → /usr/lib/systemd/system/ssh.service.
curso@dataeng:~$ sudo systemctl start ssh
curso@dataeng:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-09-09 17:05:54 UTC; 23s ago
     TriggeredBy: ● ssh.socket
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 38685 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 38686 (sshd)
      Tasks: 1 (limit: 4543)
     Memory: 1.2M (peak: 1.5M)
        CPU: 15ms
   CGroup: /system.slice/ssh.service
           └─38686 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

sep 09 17:05:54 dataeng systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
sep 09 17:05:54 dataeng sshd[38686]: Server listening on 0.0.0.0 port 22.
sep 09 17:05:54 dataeng sshd[38686]: Server listening on :: port 22.
sep 09 17:05:54 dataeng systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
curso@dataeng:~$ _
```

4. Conectar desde Windows Terminal

Abrir Windows Terminal y ejecutar:

```
ssh curso@192.168.86.130
```

Donde:

- `curso` es el usuario de Ubuntu.
- `192.168.86.130` es la IP del servidor.

La primera vez aparecerá:

```
Are you sure you want to continue connecting (yes/no)?
```

Responder:

```
yes
```

```
Símbolo del sistema - ssh cur X + v
Microsoft Windows [Versión 10.0.26200.9168]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\juanjo>
C:\Users\juanjo>ssh curso@192.168.86.130
The authenticity of host '192.168.86.130 (192.168.86.130)' can't be established.
ED25519 key fingerprint is SHA256:i6XBJY0LkHQYKA+C1onaR4M97otrdfhpRlfQrDPkmIc.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? |
```

Introducir la contraseña del usuario Ubuntu.

```
Microsoft Windows [Versión 10.0.26200.9168]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\juanjo>
C:\Users\juanjo>ssh curso@192.168.86.130
The authenticity of host '192.168.86.130 (192.168.86.130)' can't be established.
ED25519 key fingerprint is SHA256:i6XBJY0LkHQYKA+C1onaR4M97otrdfhpRlfQrDPkmIc.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.86.130' (ED25519) to the list of known hosts.
curso@192.168.86.130's password:
```

Si todo está correcto aparecerá:

```
curso@dataeng:~$
```

```
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.8.0-139-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of mié 09 sep 2026 17:10:53 UTC

System load:  0.05          Processes:            247
Usage of /:   52.8% of 13.67GB Users logged in:          1
Memory usage: 22%          IPv4 address for ens33: 192.168.86.130
Swap usage:   0%

El mantenimiento de seguridad expandido para Applications está desactivado
Se pueden aplicar 0 actualizaciones de forma inmediata.

Active ESM Apps para recibir futuras actualizaciones de seguridad adicionales.
Vea https://ubuntu.com/esm o ejecute «sudo pro status»

curso@dataeng:~$ |
```

5. Verificar que estamos trabajando en Ubuntu

Ejecutar:

```
uname -a
```

Resultado esperado:

```
Linux dataeng 6.8.0-139-generic #139-Ubuntu SMP PREEMPT_DYNAMIC Sat Aug 1 03:52:05 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
```

Probar también:

```
pwd
```

```
ls
```

```
whoami
```

```
curso@dataeng:~$ uname -a
Linux dataeng 6.8.0-139-generic #139-Ubuntu SMP PREEMPT_DYNAMIC Sat Aug 1 03:52:05 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
curso@dataeng:~$ pwd
/home/curso
curso@dataeng:~$ ls
curso@dataeng:~$ whoami
curso
curso@dataeng:~$ |
```

6. Instalar Visual Studio Code

Descargar desde:

<https://code.visualstudio.com>

Instalar normalmente en Windows.

7. Instalar la extensión Remote SSH

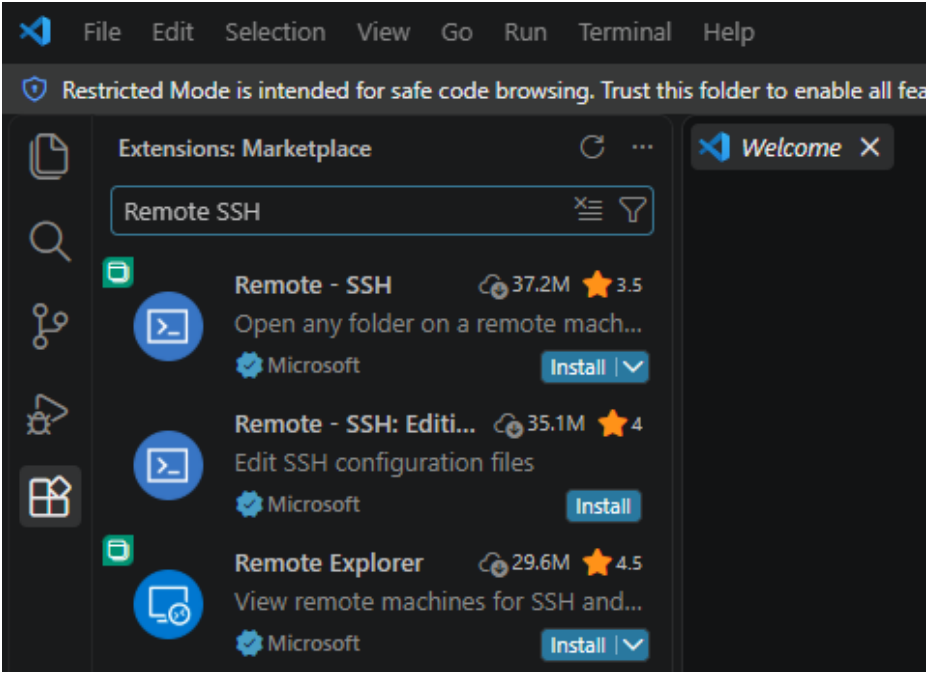
Abrir VS Code.

Ir a:

Extensiones

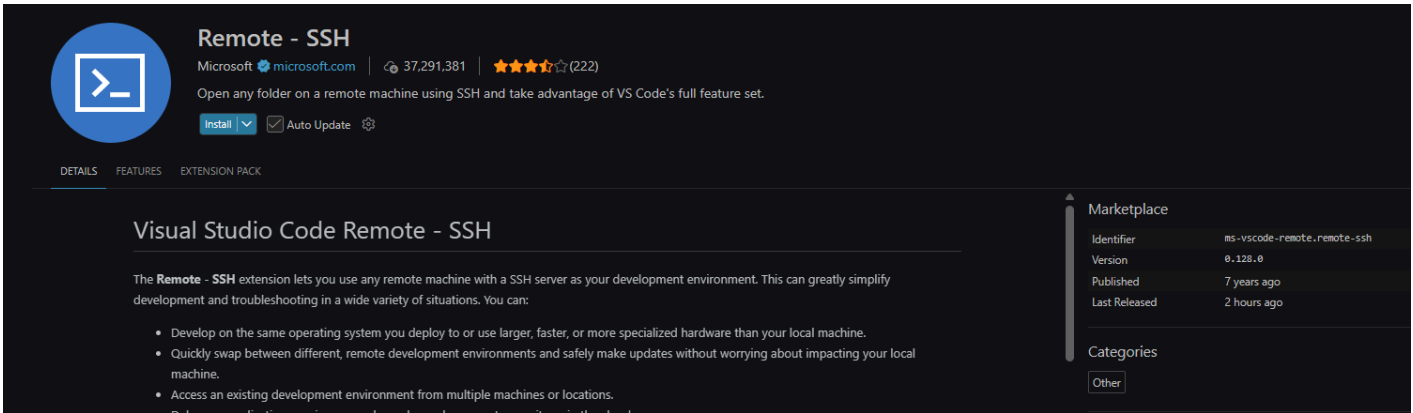
Buscar:

Remote - SSH



Instalar la extensión oficial de Microsoft:

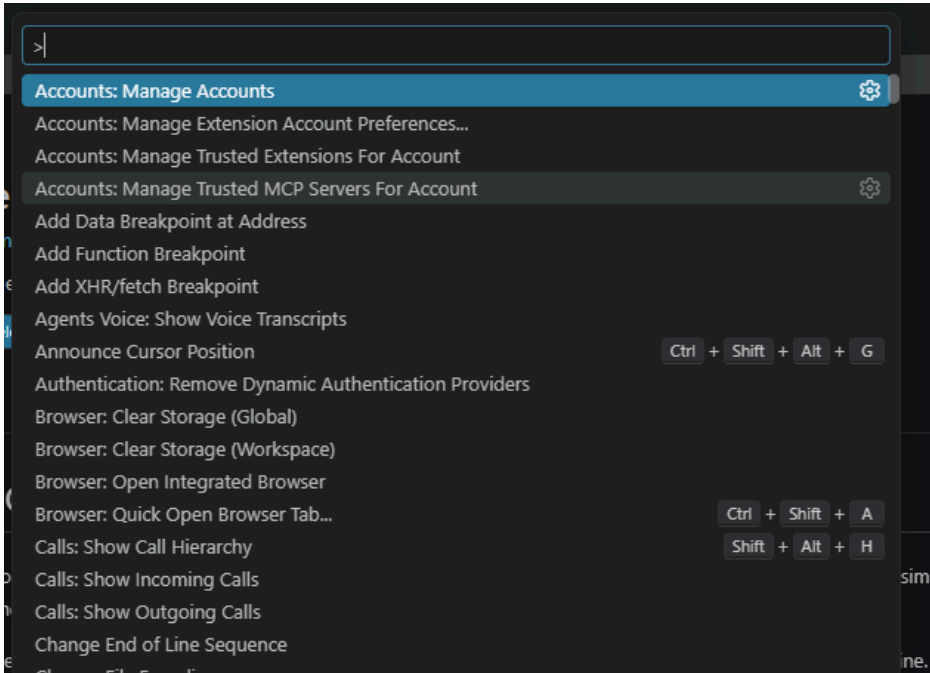
ms-vscode-remote.remote-ssh



8. Configurar la conexión SSH en VS Code

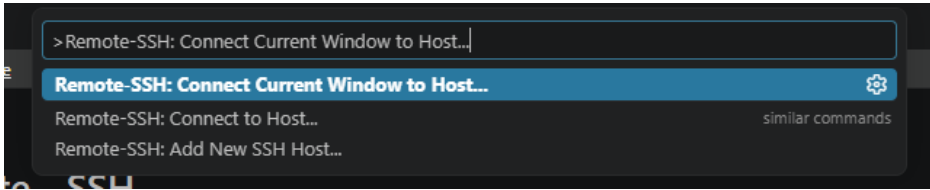
Abrir la paleta de comandos:

Ctrl + Shift + P



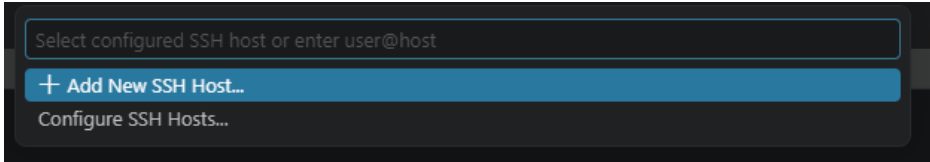
Escribir:

```
Remote-SSH: Connect Current Window to Host...
```



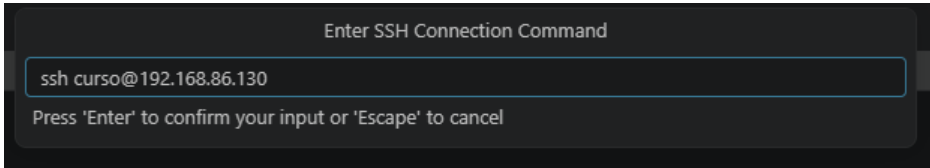
Seleccionar:

```
Add New SSH Host
```



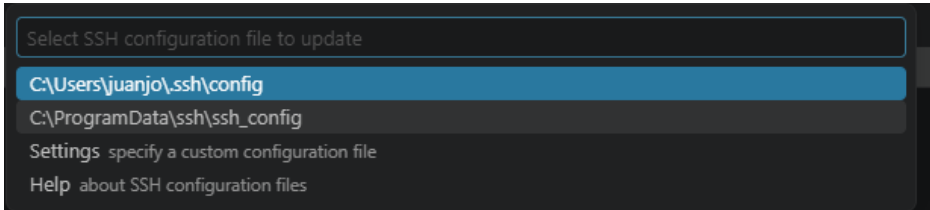
Introducir:

```
ssh curso@192.168.86.130
```



Guardar en:

```
C:\Users\juanjo\.ssh\config
```



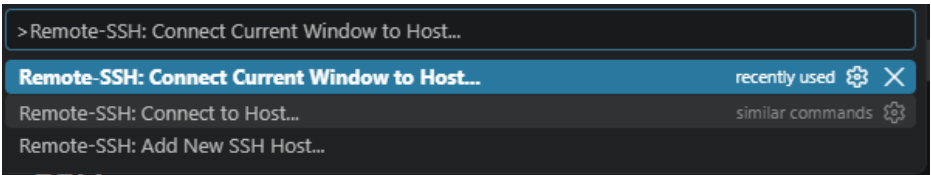
9. Conectarse a Ubuntu desde VS Code

Abrir nuevamente:

```
Ctrl + Shift + P
```

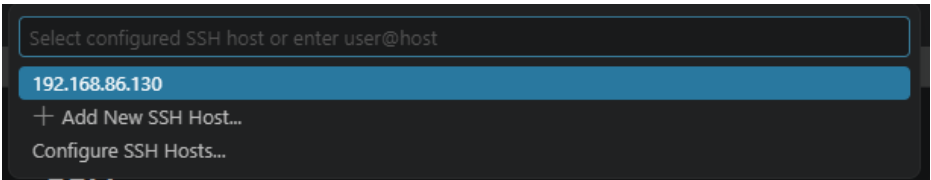
Ejecutar:

```
Remote-SSH: Connect Current Window to Host...
```

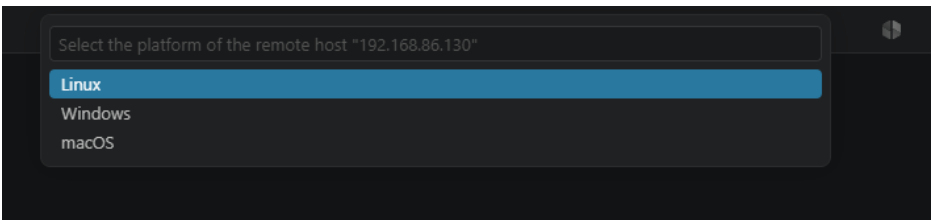


Seleccionar:

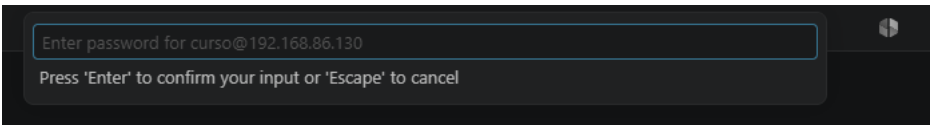
```
curso@192.168.86.130
```



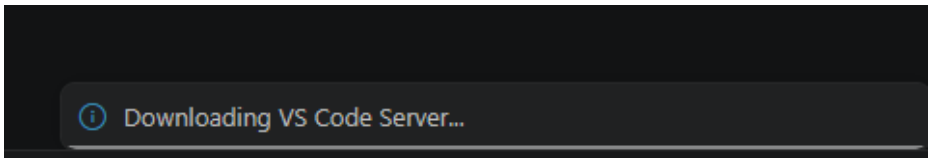
Seleccionar Linux



Introduce password:



eberías ver:

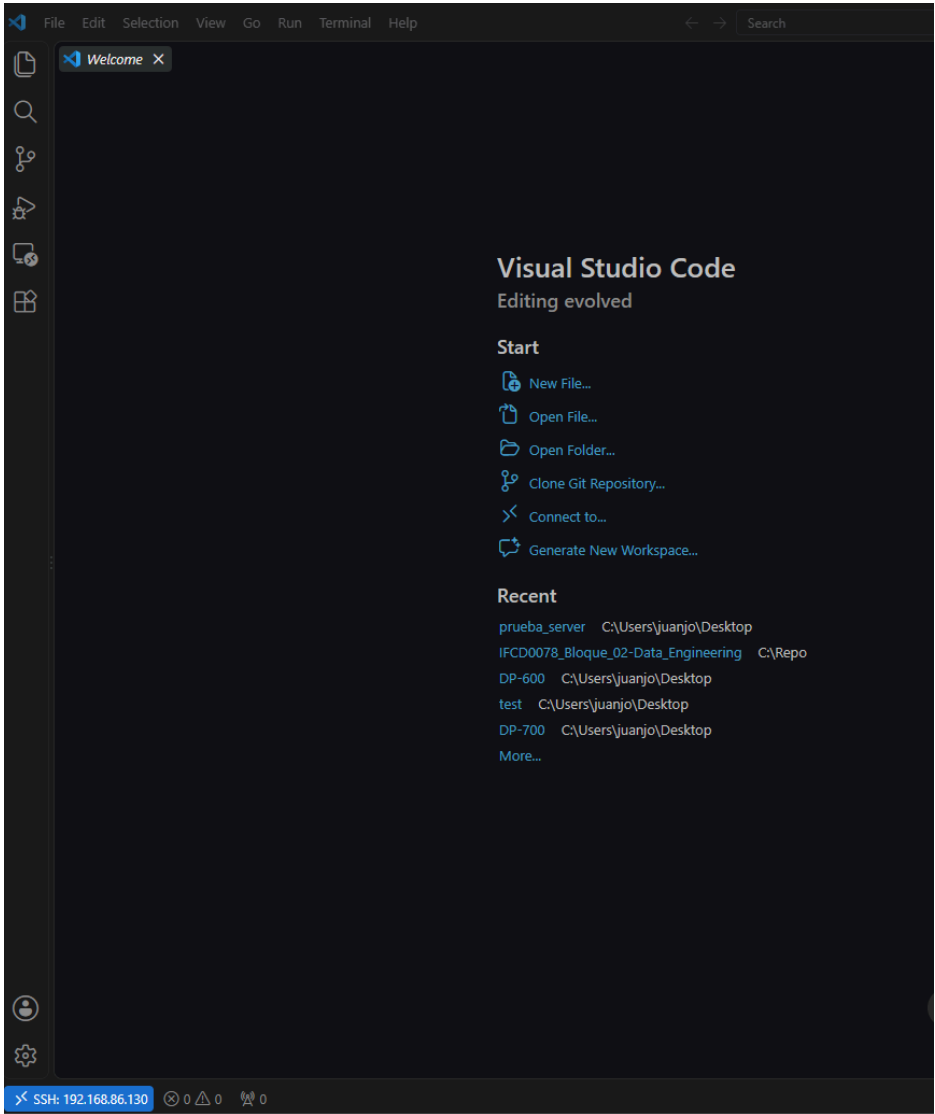


VS Code instalará automáticamente los componentes necesarios.

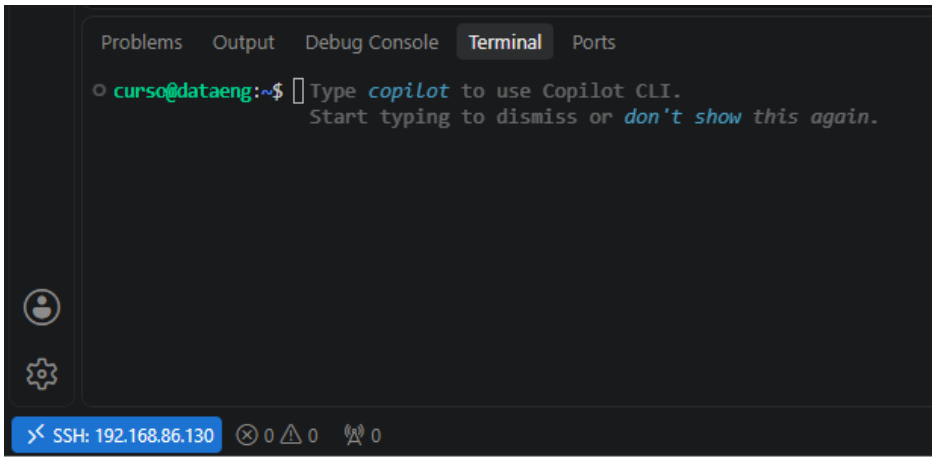
Cuando la conexión esté activa se verá:

```
SSH: 192.168.86.130
```

en la esquina inferior izquierda.



Ve a la terminal de VScode:



10. Actualizar Ubuntu

Antes de instalar Docker, escribe en la terminal:

```
sudo apt update
```

```
sudo apt upgrade -y
```

```
● curso@dataeng:~$ sudo apt update
[sudo] password for curso:
Obj:1 http://es.archive.ubuntu.com/ubuntu noble InRelease
Obj:2 http://es.archive.ubuntu.com/ubuntu noble-updates InRelease
Obj:3 http://es.archive.ubuntu.com/ubuntu noble-backports InRelease
Obj:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se pueden actualizar 4 paquetes. Ejecute «apt list --upgradable» para verlos.
● curso@dataeng:~$ sudo apt upgrade -y
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Calculando la actualización... Hecho
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios.
  libfwupd2 libgusb2
Utilice «sudo apt autoremove» para eliminarlos.
The following upgrades have been deferred due to phasing:
  python-apt-common python3-apt python3-distupgrader ubuntu-release-upgrader-core
  0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 4 no actualizados.
○ curso@dataeng:~$
```

11. Instalar dependencias necesarias para Docker

```
sudo apt install -y ca-certificates curl gnupg lsb-release
```

```
● curso@dataeng:~$ sudo apt install -y ca-certificates curl gnupg lsb-release
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
ca-certificates ya está en su versión más reciente (20260601~24.04.1).
curl ya está en su versión más reciente (8.5.0-2ubuntu10.13).
gnupg ya está en su versión más reciente (2.4.4-2ubuntu17.6).
lsb-release ya está en su versión más reciente (12.0-2).
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios.
  libfwupd2 libgusb2
Utilice «sudo apt autoremove» para eliminarlos.
 0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 4 no actualizados.
○ curso@dataeng:~$
```

12. Crear directorio para claves GPG

```
sudo install -m 0755 -d /etc/apt/keyrings
```

```
● curso@dataeng:~$ sudo install -m 0755 -d /etc/apt/keyrings
○ curso@dataeng:~$
```

13. Descargar la clave oficial de Docker

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/a
pt/keyrings/docker.gpg
```

Si te sale sobre escribir:

```
○ curso@dataeng:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
File '/etc/apt/keyrings/docker.gpg' exists. Overwrite? (y/N)
```

escribe **y** y luego Enter:

```
● curso@dataeng:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
File '/etc/apt/keyrings/docker.gpg' exists. Overwrite? (y/N) y
○ curso@dataeng:~$
```

Asignar permisos:

```
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

```
• curso@dataeng:~$ sudo chmod a+r /etc/apt/keyrings/docker.gpg
○ curso@dataeng:~$
```

14. Agregar el repositorio oficial de Docker

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
• curso@dataeng:~$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
○ curso@dataeng:~$
```

15. Actualizar repositorios

```
sudo apt update
```

```
• curso@dataeng:~$ sudo apt update
Obj:1 http://es.archive.ubuntu.com/ubuntu noble InRelease
Obj:2 http://es.archive.ubuntu.com/ubuntu noble-updates InRelease
Obj:3 http://es.archive.ubuntu.com/ubuntu noble-backports InRelease
Des:4 https://download.docker.com/linux/ubuntu noble InRelease [48,5 kB]
Obj:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Des:6 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [65,7 kB]
Descargados 114 kB en 0s (246 kB/s)
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se pueden actualizar 4 paquetes. Ejecute «apt list --upgradable» para verlos.
○ curso@dataeng:~$
```

16. Instalar Docker Engine y Docker Compose

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Deberías obtener en la salida esto (no pegar):

```
curso@dataeng:~$
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios.
  libfwupd2 libgusb2
Utilice «sudo apt autoremove» para eliminarlos.
Se instalarán los siguientes paquetes adicionales:
  docker-ce-rootless-extras pigz
Paquetes sugeridos:
  cgroupfs-mount | cgroup-lite docker-model-plugin
Se instalarán los siguientes paquetes NUEVOS:
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras docker-compose-plugin pigz
0 actualizados, 7 nuevos se instalarán, 0 para eliminar y 4 no actualizados.
```

```
Se necesita descargar 99,8 MB de archivos.
Se utilizarán 382 MB de espacio de disco adicional después de esta operación.
Des:1 http://es.archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65,6
kB]
Des:2 https://download.docker.com/linux/ubuntu noble/stable amd64 containerd.io amd64
2.3.5-1~ubuntu.24.04~noble [22,4 MB]
Des:3 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce-cli amd64
5:29.8.0-1~ubuntu.24.04~noble [17,6 MB]
Des:4 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce amd64 5:2
9.8.0-1~ubuntu.24.04~noble [24,3 MB]
Des:5 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-buildx-plugin
amd64 0.37.0-1~ubuntu.24.04~noble [17,3 MB]
Des:6 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce-rootless-e
xtras amd64 5:29.8.0-1~ubuntu.24.04~noble [10,2 MB]
Des:7 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-compose-plugi
n amd64 5.5.1-1~ubuntu.24.04~noble [8.012 kB]
Descargados 99,8 MB en 2s (61,2 MB/s)
Seleccionando el paquete containerd.io previamente no seleccionado.
(Leyendo la base de datos ... 88443 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../0-containerd.io_2.3.5-1~ubuntu.24.04~noble_amd64.deb
...
Desempaquetando containerd.io (2.3.5-1~ubuntu.24.04~noble) ...
Seleccionando el paquete docker-ce-cli previamente no seleccionado.
Preparando para desempaquetar .../1-docker-ce-cli_5%3a29.8.0-1~ubuntu.24.04~noble_amd6
4.deb ...
Desempaquetando docker-ce-cli (5:29.8.0-1~ubuntu.24.04~noble) ...
Seleccionando el paquete docker-ce previamente no seleccionado.
Preparando para desempaquetar .../2-docker-ce_5%3a29.8.0-1~ubuntu.24.04~noble_amd64.de
b ...
Desempaquetando docker-ce (5:29.8.0-1~ubuntu.24.04~noble) ...
Seleccionando el paquete pigz previamente no seleccionado.
Preparando para desempaquetar .../3-pigz_2.8-1_amd64.deb ...
Desempaquetando pigz (2.8-1) ...
Seleccionando el paquete docker-buildx-plugin previamente no seleccionado.
Preparando para desempaquetar .../4-docker-buildx-plugin_0.37.0-1~ubuntu.24.04~noble_a
md64.deb ...
Desempaquetando docker-buildx-plugin (0.37.0-1~ubuntu.24.04~noble) ...
Seleccionando el paquete docker-ce-rootless-extras previamente no seleccionado.
Preparando para desempaquetar .../5-docker-ce-rootless-extras_5%3a29.8.0-1~ubuntu.24.0
4~noble_amd64.deb ...
Desempaquetando docker-ce-rootless-extras (5:29.8.0-1~ubuntu.24.04~noble) ...
Seleccionando el paquete docker-compose-plugin previamente no seleccionado.
Preparando para desempaquetar .../6-docker-compose-plugin_5.5.1-1~ubuntu.24.04~noble_a
md64.deb ...
Desempaquetando docker-compose-plugin (5.5.1-1~ubuntu.24.04~noble) ...
Configurando docker-buildx-plugin (0.37.0-1~ubuntu.24.04~noble) ...
Configurando containerd.io (2.3.5-1~ubuntu.24.04~noble) ...
Created symlink /etc/systemd/system/multi-user.target.wants/containerd.service → /usr/
lib/systemd/system/containerd.service.
Configurando docker-compose-plugin (5.5.1-1~ubuntu.24.04~noble) ...
Configurando docker-ce-cli (5:29.8.0-1~ubuntu.24.04~noble) ...
Configurando pigz (2.8-1) ...
Configurando docker-ce-rootless-extras (5:29.8.0-1~ubuntu.24.04~noble) ...
Configurando docker-ce (5:29.8.0-1~ubuntu.24.04~noble) ...
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/
systemd/system/docker.service.
Created symlink /etc/systemd/system/sockets.target.wants/docker.socket → /usr/lib/syst
emd/system/docker.socket.
Procesando disparadores para man-db (2.12.0-4build2) ...
Scanning processes...
```

```
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
curso@dataeng:~$
```

17. Verificar la instalación

Versión de Docker:

```
docker --version
```

```
curso@dataeng:~$ docker --version
Docker version 29.8.0, build 88096ef
curso@dataeng:~$
```

Versión de Docker Compose:

```
docker compose version
```

```
curso@dataeng:~$ docker compose version
Docker Compose version v5.5.1
curso@dataeng:~$
```

Estado del servicio:

```
sudo systemctl status docker
```

Debe aparecer:

```
Active: active (running)
```

```
curso@dataeng:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-09-09 17:32:38 UTC; 1min 53s ago
     TriggeredBy: ● docker.socket
   Docs: https://docs.docker.com
   Main PID: 79695 (dockerd)
    Tasks: 9
   Memory: 27.7M (peak: 28.2M)
      CPU: 408ms
   CGroup: /system.slice/docker.service
           └─79695 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

sep 09 17:32:37 dataeng dockerd[79695]: time="2026-09-09T17:32:37.779105183Z" level=info msg="Restoring containers: start."
sep 09 17:32:37 dataeng dockerd[79695]: time="2026-09-09T17:32:37.795581234Z" level=info msg="Deleting nftables IPv4 rules" error="running nft: /dev/stdin:1:17-30: Error: Could not process rule: No such file or directory\ndelete table ip docker-bridges\n"
sep 09 17:32:37 dataeng dockerd[79695]: time="2026-09-09T17:32:37.802954402Z" level=info msg="Deleting nftables IPv6 rules" error="running nft: /dev/stdin:1:18-31: Error: Could not process rule: No such file or directory\ndelete table ip6 docker-bridges\n"
sep 09 17:32:38 dataeng dockerd[79695]: time="2026-09-09T17:32:38.129055832Z" level=info msg="Loading containers: done."
sep 09 17:32:38 dataeng dockerd[79695]: time="2026-09-09T17:32:38.136006193Z" level=info msg="Docker daemon" commit=3ce5872 containerd-snapshotter=true storage-driver=overlayfs version=29.8.0
sep 09 17:32:38 dataeng dockerd[79695]: time="2026-09-09T17:32:38.136079000Z" level=info msg="Initializing buildkit"
sep 09 17:32:38 dataeng dockerd[79695]: time="2026-09-09T17:32:38.271944065Z" level=info msg="Completed buildkit initialization"
sep 09 17:32:38 dataeng dockerd[79695]: time="2026-09-09T17:32:38.279509982Z" level=info msg="Daemon has completed initialization"
sep 09 17:32:38 dataeng dockerd[79695]: time="2026-09-09T17:32:38.279638185Z" level=info msg="API listen on /run/docker.sock"
sep 09 17:32:38 dataeng systemd[1]: Started docker.service - Docker Application Container Engine.
lines 1-22/22 (END)
```

pulsar q para salir del END:

```
curso@dataeng:~$
```

18. Permitir usar Docker sin sudo

Agregar el usuario al grupo docker:

```
sudo usermod -aG docker curso
```

```
curso@dataeng:~$ sudo usermod -aG docker curso
curso@dataeng:~$
```

Cerrar sesión:

```
exit
```

Volver a conectarse:

```
ssh curso@192.168.86.130
```

19. Probar Docker

Ejecutar:

```
docker run hello-world
```

Si todo funciona correctamente aparecerá:

```
Hello from Docker!
```

Si te sale este error:

```
curso@dataeng:~$ docker run hello-world
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
```

esto es que el usuario aún no pertenece al grupo `docker` en la sesión actual.

Ejecuta:

```
groups
```

```
curso@dataeng:~$ groups
curso adm cdrom sudo dip plugdev lxd
curso@dataeng:~$
```

Si no aparece `docker` en la lista, es normal. Ahora ejecuta:

```
sudo usermod -aG docker curso
```

```
curso@dataeng:~$ sudo usermod -aG docker curso
[sudo] password for curso:
```

Después verifica:

```
grep docker /etc/group
```

Deberías ver algo parecido a:

```
curso@dataeng:~$ grep docker /etc/group
docker:x:988:curso
curso@dataeng:~$
```

Muy importante

Los cambios de grupo **no se aplican a la sesión actual**.

Debes salir completamente de la sesión SSH:

```
exit
```

Vuelve a conectarte desde VS Code:

Pulsa: `Ctrl + Shift + P`

Ejecuta:

```
Remote-SSH: Connect Current Window to Host...
```

Selecciona la ip e introduces el password.

Ejecuta:

```
newgrp docker
```

Después comprueba:

```
groups
```

Ahora debería aparecer:

```
curso@dataeng:~$ newgrp docker
curso@dataeng:~$ groups
docker adm cdrom sudo dip plugdev lxd curso
curso@dataeng:~$
```

Ejecuta ahora:

```
docker run hello-world
```

La primera vez Docker descargará la imagen desde Docker Hub y deberías ver algo parecido a:

```
curso@dataeng:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5dd0d3e6e255913fc30f90b9f2b1d359cc2cbdb48090cc4b65f1676e203243cc
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

curso@dataeng:~$ █
```

Comprobaciones finales:

Ejecuta estos comandos:

```
docker --version
```

```
curso@dataeng:~$ docker --version
Docker version 29.8.0, build 88096ef
curso@dataeng:~$ █
```

```
docker compose version
```

```
curso@dataeng:~$ docker --version
Docker version 29.8.0, build 88096ef
curso@dataeng:~$ docker compose version
Docker Compose version v5.5.1
curso@dataeng:~$ █
```

```
docker ps -a
```

Deberías obtener algo similar a:

```
curso@dataeng:~$ docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS      PORTS          NAMES
7e4ccd9c8b64   hello-world "/hello"                2 minutes ago Exited (0) 2 minutes ago          reverent_banzai
curso@dataeng:~$ █
```

Prueba adicional:

Descarga y ejecuta Nginx:

```
docker run -d --name nginx-prueba -p 80:80 nginx
```

Comprueba que está funcionando:

```
docker ps
```

Verás algo parecido a:

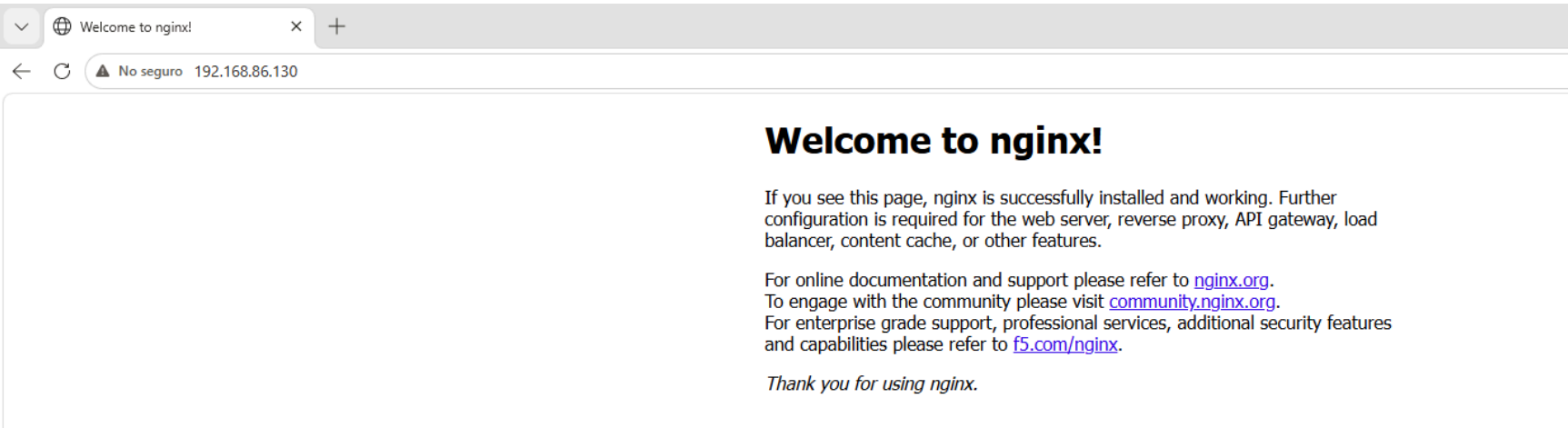
```
curso@dataeng:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS      PORTS          NAMES
da44700e3099   nginx     "/docker-entrypoint..." 6 seconds ago Up 5 seconds  0.0.0.0:80->80/tcp, [::]:80->80/tcp  nginx-prueba
curso@dataeng:~$ █
```

Abre en tu navegador de Windows:

```
http://192.168.86.130
```

(donde `192.168.86.130` es la IP que vimos antes). Si aparece la página de bienvenida de Nginx, tendrás confirmación de que:

- ✓ Ubuntu Server funciona
- ✓ VMware funciona
- ✓ Red configurada correctamente
- ✓ Docker Engine funciona
- ✓ Descarga de imágenes desde Docker Hub funciona
- ✓ Publicación de puertos funciona



20. Instalacion de Kubernetes

Vamos a verificar que nuestro ubuntu server trae microk8s.

1. Verificar que MicroK8s está funcionando

Ejecuta:

```
microk8s status
```

2. Verificar el estado del nodo

```
microk8s kubectl get nodes
```

Resultado esperado:

NAME	STATUS	ROLES	AGE	VERSION
dataeng	Ready	<none>	25h	v1.35.6

Puntos importantes:

- STATUS = Ready
- Debe aparecer el nodo `dataeng`

3. Ver información del clúster

```
microk8s kubectl cluster-info
```

Resultado esperado:

```
Kubernetes control plane is running at https://127.0.0.1:16443
```

4. Ver todos los pods de Kubernetes

```
microk8s kubectl get pods -A
```

Resultado esperado:

NAMESPACE	NAME	STATUS
kube-system	calico-kube-controllers-xxxxx	Running
kube-system	calico-node-xxxxx	Running
kube-system	coredns-xxxxx	Running
kube-system	hostpath-provisioner-xxxxx	Running
kube-system	metrics-server-xxxxx	Running

Todos deben aparecer en:

```
STATUS = Running
```

5. Ver todos los recursos del clúster

```
microk8s kubectl get all -A
```

Permite ver:

- Pods
- Services
- Deployments
- ReplicaSets
- DaemonSets

6. Verificar DNS

```
microk8s status
```

Debe aparecer:

```
dns
```

como addon habilitado.

También:

```
microk8s kubectl get pods -n kube-system
```

Debe aparecer:

```
coredns
```

en estado:

```
Running
```

7. Verificar almacenamiento persistente

Ver addon:

```
microk8s status
```

Debe aparecer:

```
hostpath-storage
```

habilitado.

Ver StorageClass:

```
microk8s kubectl get storageclass
```

Resultado esperado:

```
microk8s-hostpath (default)
```

8. Verificar Metrics Server

Ver addon:

```
microk8s status
```

Debe aparecer:

```
metrics-server
```

habilitado.

Comprobar consumo de recursos:

```
microk8s kubectl top nodes
```

Ejemplo:

NAME	CPU(cores)	CPU(%)	MEMORY(bytes)	MEMORY(%)
dataeng	423m	21%	2185Mi	57%

9. Verificar Helm

```
microk8s status
```

Debe aparecer:

```
helm
helm3
```

habilitados.

Comprobar versión:

```
microk8s helm3 version
```

10. Verificar puertos de Kubernetes

```
sudo ss -tulpn | grep 16443
```

Resultado esperado:

```
*:16443
```

Corresponde al API Server de Kubernetes.

21. Configuración recomendada para las clases

- Añadir alias permanentes:

```
echo "alias kubectl='microk8s kubectl'" >> ~/.bashrc
```

```
echo "alias k='microk8s kubectl'" >> ~/.bashrc
```

- Aplicar cambios:

```
source ~/.bashrc
```

- Verificar:

```
kubectl get nodes
```

o

```
k get nodes
```

Resultado esperado:

NAME	STATUS	ROLES	AGE	VERSION
dataeng	Ready	<none>	...	

- Verificar que Metrics Server funciona

```
kubectl top nodes
```

Resultado esperado:

NAME	CPU (cores)	CPU%	MEMORY (bytes)	MEMORY%
dataeng	...			

- Verificar el almacenamiento persistente

```
kubectl get storageclass
```

Resultado esperado:

```
microk8s-hostpath (default)
```

- Verificar todos los pods del sistema

```
kubectl get pods -A
```

Todos los pods deben aparecer como:

```
Running
```

Verificación final del laboratorio

```
kubectl get nodes
```

```
kubectl get pods -A
```

```
kubectl get storageclass
```

```
kubectl get storageclass
```

```
kubectl top nodes
```

```
kubectl cluster-info
```